ASSIGNMENT 1.5(BATCH-30)

2303A51930

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

❖ Scenario

You are developing a basic text-processing utility for a messaging application.

❖ Task Description

Use GitHub Copilot to generate a Python program that:

➢ Reverses a given string

➢ Accepts user input

➢ Implements the logic directly in the main code

➢ Does not use any user-defined functions

CODE:

```
# Accept user input
user_string = input("Enter a string to reverse: ")
# Reverse the string directly using slicing
reversed_string = user_string[::-1]
# Display the result
print("Original string:", user_string)
print("Reversed string:", reversed_string)
```

OUTPUT:

```
PS D:\AI Assisted coding> ^C
PS D:\AI Assisted coding>
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12
.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher' '638
08' '--' 'D:\AI Assisted coding\.vscode\1.5(TASK-1).py'
Enter a string to reverse: PAVAN
Original string: PAVAN
Reversed string: NAVAP
PS D:\AI Assisted coding>
```

Task 2: Efficiency & Logic Optimization (Readability Improvement)

❖ Scenario

The code will be reviewed by other developers.

❖ Task Description

Examine the Copilot-generated code from Task 1 and improve it by:

➢ Removing unnecessary variables

➢ Simplifying loop or indexing logic

➢ Improving readability

➢ Use Copilot prompts like:

▪ “Simplify this string reversal code”

▪ “Improve readability and efficiency”

CODE:

```
# ============================================
# String Reversal Program
# ============================================
# Step 1: Accept user input
print("=" * 45)
print("STRING REVERSAL")
print("=" * 45)
user_string = input("\nEnter a string to reverse: ")
# Step 2: Reverse the string using slicing
reversed_string = user_string[::-1]
# Step 3: Display the results
print("\n" + "-" * 45)
print(f"Original string:  {user_string}")
print(f"Reversed string:  {reversed_string}")
print("-" * 45)
```

OUTPUT:

```
.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher' '529
30' '--' 'D:\AI Assisted coding\.vscode\1.5(TASK-1).py'
============================================
STRING REVERSAL
============================================

Enter a string to reverse: PAVAN

--------------------------------------------
Original string:  PAVAN
Reversed string:  NAVAP
--------------------------------------------
PS D:\AI Assisted coding>
```

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

❖ Scenario

The string reversal logic is needed in multiple parts of an application.

❖ Task Description

Use GitHub Copilot to generate a function-based Python program that:

➢ Uses a user-defined function to reverse a string

➢ Returns the reversed string

➢ Includes meaningful comments (AI-assisted)

CODE:

```
# ============================================

# String Reversal Program with User-Defined Function

# ============================================

def reverse_string(input_str):

    """

   This function reverses a given string using Python's slicing technique.

   Parameters:

    -----------

    input_str (str): The string to be reversed

       Returns:

    --------

    str: The reversed version of the input string

    Logic:

    ------

Uses the slicing operator [::-1] which:
```

```
    - Starts from the end of the string
    - Goes backwards with a step of -1
    - Returns the complete reversed string
    """
 # Reverse the string using slice notation (start:stop:step)
 # [::-1] means: start from end, go to beginning, step backwards by 1
    reversed_str = input_str[::-1]
 # Return the reversed string to the caller
    return reversed_str
# Main Program Execution
# ==========================================
# Display program header with visual separators
print("=" * 50)
print("STRING REVERSAL PROGRAM")
print("=" * 50)
# Step 1: Accept input from user
# The input() function prompts the user for a string
user_input = input("\nEnter a string to reverse: ")
# Step 2: Call the reverse_string function
# Pass the user input to the function and store the returned value
reversed_result = reverse_string(user_input)
# Step 3: Display the results
# Show both original and reversed strings with clear formatting
print("\n" + "-" * 50)
print(f"Original string:  {user_input}")
print(f"Reversed string:  {reversed_result}")
print("-" * 50)
```

OUTPUT:

```
.exe' 'c:\Users\vinee\.vscode\extensions\ms-python.debugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher' '514
10' '--' 'D:\AI Assisted coding\.vscode\1.5(TASK-1).py'
==================================================
STRING REVERSAL PROGRAM
==================================================

Enter a string to reverse: PAVAN

--------------------------------------------------
Original string:  PAVAN
Reversed string:  NAVAP
--------------------------------------------------
PS D:\AI Assisted coding>
```

Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs Without Functions)

❖ Scenario

You are asked to justify design choices during a code review.

❖ Task Description

Compare the Copilot-generated programs:

➢ Without functions (Task 1)

➢ With functions (Task 3)

Analyze them based on:

➢ Code clarity

➢ Reusability

➢ Debugging ease

➢ Suitability for large-scale applications

COMPARISON TABLE

Criteria	Code 1: Without User-Defined Function	Code 2: With User-Defined Function
Code Clarity	Very simple and short. Easy to understand for beginners, but logic and input/output are mixed together.	More structured and well-organized. Logic is separated from input/output, improving readability.
Reusability	Not reusable. The reversal logic is written directly in the main code and cannot be reused without copying.	Highly reusable. The reverse_string() function can be reused in other programs or modules.
Debugging Ease	Easy for small programs, but difficult as the code grows because everything is in one place.	Easier to debug. Errors can be isolated inside the function without affecting the rest of the program.

Criteria	Code 1: Without User-Defined Function	Code 2: With User-Defined Function
Suitability for Larger-Scale Applications	Not suitable. Lack of modularity makes the code hard to maintain and scale.	Very suitable. Modular design supports maintenance, testing, and expansion in large applications.
Maintainability	Low maintainability. Any change requires editing the main logic directly.	High maintainability. Changes can be made inside the function without affecting other code.
Readability for Team Projects	Poor for team environments due to lack of structure.	Good for collaborative development due to clear separation of concerns.
Performance	Slightly faster for very small scripts due to no function call overhead (negligible difference).	Slight overhead due to function calls, but insignificant in real applications.
Assignment Compliance (Task 1)	Fully compliant with Task 1 requirements.	Not compliant, because it uses a user-defined function.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different

Algorithmic Approaches to String Reversal)

❖ Scenario

Your mentor wants to evaluate how AI handles alternative logic paths.

❖ Task Description

Prompt GitHub Copilot to generate:

➢ A loop-based string reversal approach

➢ A built-in / slicing-based string reversal approach

CODE:

```
# ==========================================
# String Reversal Program: Two Approaches
# ==========================================

# APPROACH 1: LOOP-BASED STRING REVERSAL
# ==========================================

def reverse_string_loop(input_str):
    """
    This function reverses a string using a loop-based approach.
```

```
    Parameters:
    -----------
    input_str (str): The string to be reversed

    Returns:
    --------
    str: The reversed version of the input string

    Logic:
    ------
    Iterates through the string from the end to the beginning:
    - Initialize an empty result string
    - Loop from the last character index to 0
    - Concatenate each character to the result
    - Return the concatenated reversed string
    """

    # Initialize an empty string to store the reversed result
    reversed_str = ""

    # Loop from the last character index (len-1) down to 0
    # range(len(input_str)-1, -1, -1) creates: last_index, ..., 1, 0
    for i in range(len(input_str) - 1, -1, -1):
        # Append each character starting from the end
        reversed_str += input_str[i]

    # Return the completely reversed string
    return reversed_str

# APPROACH 2: SLICING-BASED STRING REVERSAL (BUILT-IN)
# ==========================================

def reverse_string_slicing(input_str):
    """
    This function reverses a string using Python's slicing technique.

    Parameters:
    -----------
    input_str (str): The string to be reversed

    Returns:
    --------
    str: The reversed version of the input string

    Logic:
    ------
    Uses the slicing operator [::-1] which:
    - Starts from the end of the string (default start position)
    - Goes to the beginning (default stop position)
    - Steps backwards by 1 with the negative step value
    - Returns the complete reversed string efficiently
```

```
    """

    # Reverse the string using slice notation (start:stop:step)
    # [::-1] means: start from end, go to beginning, step backwards by 1
    # This is the most Pythonic and efficient way to reverse a string
    reversed_str = input_str[::-1]

    # Return the reversed string to the caller
    return reversed_str

# MAIN PROGRAM EXECUTION
# ==============================================

# Display program header with visual separators
print("=" * 60)
print("STRING REVERSAL PROGRAM - TWO APPROACHES")
print("=" * 60)

# Step 1: Accept input from user
# The input() function prompts the user to enter a string
user_input = input("\nEnter a string to reverse: ")

# Step 2a: Reverse using LOOP-BASED approach
# Pass the user input to the loop-based function
reversed_loop = reverse_string_loop(user_input)

# Step 2b: Reverse using SLICING-BASED approach
# Pass the user input to the slicing-based function
reversed_slicing = reverse_string_slicing(user_input)

# Step 3: Display the results
# Show original and both reversed versions with clear formatting
print("\n" + "-" * 60)
print(f"Original string:         {user_input}")
print(f"Reversed (Loop Method):    {reversed_loop}")
print(f"Reversed (Slicing Method): {reversed_slicing}")
print("-" * 60)
```

OUTPUT:

```
10' '--' 'D:\AI Assisted coding\.vscode\1.5(TASK-1).py'
============================================================
STRING REVERSAL PROGRAM - TWO APPROACHES
============================================================

Enter a string to reverse: PAVAN

------------------------------------------------------------
Original string:           PAVAN
Reversed (Loop Method):    NAVAP
Reversed (Slicing Method): NAVAP
------------------------------------------------------------
PS D:\AI Assisted coding>
```